

Übung 4

Mustererkennung und Maschinelles Lernen – SS 2026

Frist: 24.06.2026

Allgemeine Hinweise

- Nutzen Sie bitte das beigelegte Notebook als Vorlage.
- Für die Programmieraufgaben empfehlen wir [Google Colab](#) (keine Installation nötig, kostenlose GPU). Als Framework verwenden wir TensorFlow/[Keras](#).
- Für Aufgabe 2 könnte folgender Keras-Guide hilfreich sein: [Transfer learning & fine-tuning](#)

Aufgabe 1: Feature Maps und Parameteranzahl

Betrachten Sie das folgende kleine CNN für die Klassifikation von Farbbildern in 5 Klassen. Alle Faltungsschichten verwenden eine ReLU-Aktivierung, kein Padding (P = 0) außer wo angegeben, und Stride S = 1 außer wo angegeben.

Schicht	Typ	Filter / Neuronen	Kernel	Stride	Padding
Input	–	–	–	–	–
Conv 1	Conv2D	32	3×3	1	1 (same)
Pool 1	MaxPool	–	2×2	2	0
Conv 2	Conv2D	64	3×3	1	0
Pool 2	MaxPool	–	2×2	2	0
Flatten	Flatten	–	–	–	–
Dense 1	Dense	128	–	–	–
Dense 2	Dense	5 (Softmax)	–	–	–

Die Eingabe besteht aus Bildern der Größe 64×64×3.

- Berechnen Sie für jede Schicht die Ausgabegröße.
- Berechnen Sie die Anzahl der lernbaren Parameter je Schicht sowie die Gesamtzahl. Berücksichtigen Sie auch die Biases.
- Welche einzelne Schicht enthält mit Abstand die meisten Parameter? Erklären Sie, woran das liegt. Was bedeutet diese Beobachtung für sehr große Netze wie VGG16 (138M Parameter), und welche Designidee moderner Architekturen (Stichwort: Global Average Pooling) adressiert genau dieses Problem?

Aufgabe 2: Merkmalsextraktion und Finetuning

In dieser Aufgabe nutzen Sie ein auf ImageNet vortrainiertes Netz, um mit wenig Daten und ohne Training von Grund auf einen guten Bildklassifikator zu bauen. Sie können Google Colab mit aktivierter GPU nutzen (Laufzeit → Laufzeittyp ändern → GPU).

- Laden Sie einen kleinen Bilddatensatz mit mehreren Klassen; in der Vorlage wird `tf_flowers` verwendet. Teilen Sie in Trainings-, Validierungs- und Testdaten auf. Vollziehen Sie den Code in der Vorlage nach und schauen Sie sich die enthaltenen Daten an. Warum wird hier neben einem Validierungsdatsatz noch ein Test-Set erzeugt?
- Laden Sie ein vortrainiertes Basismodell (z. B. `MobileNetV2` oder `VGG16`) aus `keras.applications` ohne den Klassifikationskopf (`include_top=False`). Frieren Sie die Faltungsbasis ein (`base_model.trainable = False`).
- Setzen Sie einen neuen Klassifikator auf der Basis auf (z. B. `GlobalAveragePooling2D` + `Dense`). Trainieren Sie nur diesen neuen Teil (Merkmalsextraktion) und plotten Sie Trainings- und Validierungsgenauigkeit über die Epochen.
- Fine Tuning: Geben Sie nun die obersten Faltungsschichten des Basismodells frei und trainieren Sie mit einer *sehr kleinen* Lernrate (z. B. $1e-5$) weiter. Vergleichen Sie die Kurven mit (c). Wann beobachten Sie Anzeichen von Overfitting (vgl. Vorlesung: „Gefahr des schnellen Overfittings“)?
- Diskutieren Sie: Warum trainiert man beim Fine Tuning mit einer so kleinen Lernrate? Warum gibt man nur die *oberen* Schichten frei und nicht die unteren?

Hinweis: Arbeiten Sie in dieser Aufgabe nur mit der Metrik „accuracy“ und den Verlustkurven. Differenziertere Bewertungsmetriken folgen in einer späteren Vorlesung.

Aufgabe 3: Embeddings als Repräsentationen

Ein Feature-Extractor bildet jedes Bild auf einen Vektor (Embedding) ab und betreibt damit u. U. eine Form der Dimensionalitätsreduktion. In dieser Aufgabe prüfen Sie, ob diese Embeddings *semantisch sinnvoll* sind: Bleiben sich ähnliche Ziffern auch im Embedding-Raum nahe?

Als Feature-Extractor dient ein LeNet-5, eines der ersten erfolgreichen CNNs. Das Training ist im Notebook vollständig vorgegeben und dauert nur wenige Minuten; Ihre Aufgabe beginnt danach mit dem Extrahieren der Embeddings.

- Laden Sie MNIST (`keras.datasets.mnist`) und wählen Sie eine zufällige, klassenbalancierte Teilmenge von ca. 2000–3000 Bildern (das hält t-SNE schnell). Normalisieren Sie die Pixel auf $[0, 1]$.
- Erstellen Sie aus dem trainierten Netz ein Teilmodell bis zur Embedding-Schicht und berechnen Sie in *einem* Vorwärtsthroughlauf (`extractor.predict(...)`) pro Bild ein Embedding. Welche Dimension hat ein einzelnes Embedding?
- Baseline: Wenden Sie t-SNE (`sklearn.manifold.TSNE`) direkt auf die flachgemachten Rohpixel an (jedes Bild als 784-dim. Vektor) und stellen Sie das Ergebnis als 2D-Streudiagramm dar, eingefärbt nach der wahren Ziffer.
- Wenden Sie t-SNE nun auf die Embeddings aus (b) an und visualisieren Sie analog.

Vergleichen Sie beide Plots: In welchem Fall bilden die Ziffern klarere, besser getrennte Cluster? Was schließen Sie daraus über die Qualität der gelernten Repräsentation?

- e) Cluster quantifizieren: Wenden Sie K-Means (`sklearn.cluster.KMeans`) mit 10 Clustern auf die Embeddings an. Da Sie die wahren Labels kennen, können Sie die Übereinstimmung messen (z. B. mit dem Adjusted Rand Index, `sklearn.metrics.adjusted_rand_score`). Welche Ziffern werden vermutlich verwechselt? Begründen Sie.

Hinweis zur t-SNE-Perplexität: Experimentieren Sie mit verschiedenen Werten (z. B. 5, 30, 50) denken Sie an die Tücken bei der [Interpretation von t-SNE-Ergebnissen](#).